Part 1 Introduction

This lab focuses on designing and implementing a two-wide superscalar in-order RISC-V processor capable of executing up to two instructions per cycle. Building on the single-issue design from previous labs, this project integrates dual instruction fetch, steering logic, and a scoreboard to manage instruction dependencies and functional unit assignments between heterogeneous pipelines. The main objectives are to explore the principles of superscalar architecture, implement hazard detection and bypass mechanisms using a scoreboard, and evaluate performance improvements in terms of IPC and cycle count. Through progressive development from dual-fetch to full dual-issue execution, the lab provides practical experience in balancing complexity, correctness, and efficiency in modern processor design.

Part 2 Design-I

(1) Two-Wide Superscalar Processor with Dual Fetch, but Single Issue

Describe implementation:

The revised control unit maintains a seven-stage dual-fetch pipeline while modularizing the control flow of bubble propagation, bypass selection, instruction queueing, and fetch stalling.A unified helper function, next_bubble_bit(squash_i, stall_i, bubble_q_i), governs bubble updates across all stages: squash has the highest priority, followed by stall, and only when neither is asserted does the previous bubble propagate forward. This eliminates hidden priority inconsistencies that arose from each stage handling stall/squash separately.

A family of generic selectors (sel1, sel2, … sel32) abstracts A/B lane multiplexing, replacing repetitive ternary expressions with structured, width-agnostic functions. This ensures safer and clearer signal routing between pipeline stages (D, X, and W).

Two new priority-based bypass pickers, pick_am_byp and pick_bm_byp, dynamically choose the operand source from the nearest valid producer among X0–X3 and W stages, ensuring minimal latency and maximum parallel issue efficiency.

Finally, the Fetch stage adds precise control for instruction queue invalidation: when squash_Fhl is asserted, all instruction queue valid bits are cleared immediately. The stall condition for Fetch is also refined to assign stall_Fhl = (stall_Dhl && !(inst_val_Dhl && brj_taken_Dhl));allowing Fetch to continue when a branch is taken, preventing unnecessary stalls and ensuring the PC updates correctly in the same cycle.

Justify design decisions:

These changes aim to minimize data dependency latency and reduce control recovery delay while ensuring consistent timing across the superscalar pipeline.

Centralizing bubble control via next_bubble_bit enforces a pipeline-wide invariant: Flush always overrides stal*l.* This guarantees no “ghost instructions” survive a squash, resolving the subtle one-cycle lag between stages observed in the original design.

The closest-first logic in pick_am_byp and pick_bm_byp follows the hardware principle that the most recently executed stage provides the earliest available data path. This eliminates unnecessary delay caused by selecting older, farther bypass sources when multiple matches exist.

The new conditional fetch stall ensures that when the Decode stage confirms a branch-taken event, Fetch immediately resumes issuing from the new PC rather than waiting for a hazard stall to clear. In a dual-issue pipeline, this saves one full cycle for both lanes, directly improving IPC (instructions per cycle).

Collectively, these design decisions balance correctness and throughput by tightening control timing at the microarchitectural level.

Part 2 Design-II

(2) Two-Wide Superscalar Processor

Describe implemented :

In this design, the Decode stage forms an A/B instruction pair and uses a pair-aware steering mechanism to determine whether both instructions can be issued simultaneously and assigned to separate lanes. The control logic coordinates same-cycle dual issue through request_second_issue_Dhl and issue_second_pending_Dhl, while the scoreboard's stall_0_hazard and stall_1_hazard signals manage issue permission for the second instruction.

The scoreboard tracks the in-flight state and stage tag of each destination register across stages X0–X3 and W. It performs two types of dependency checks: same-cycle pair hazards (RAW/WAW between A and B, excluding x0 writes) and cross-cycle in-flight hazards (dependencies on unready load or mul/div results). The bypass network follows a nearest-producer-first rule (X0→X3→W) and supports immediate writeback recognition (e.g., hitWB00) to supply the earliest available operands. For unready multi-cycle operations, bypassing is disabled to preserve correctness.

In control flow,stall_Fhl = stall_Dhl && !(inst_val_Dhl && brj_taken_Dhl);allows a taken branch to override normal Decode stalls for immediate PC update, while squash_Fhl clears instruction queues to prevent stale reissues. The unified bubble propagation rule (flush > stall > hold) ensures consistent control priority across the pipeline. Overall, this refined control design achieves stable and efficient dual-issue execution without altering the original datapath.

Justify design decisions:

The design prioritizes correctness with minimal latency as its core principle.By performing pair-aware checks during the Decode stage, same-cycle RAW and WAW hazards can be resolved immediately, preventing additional wiring and timing complexity in the forwarding network. For example:If pair_waw = (dstA == dstB && rd ≠ x0), the control logic directly sets stall_1_hazard = 1, preventing both instructions from writing to the same register.If pair_raw1 (B reads A) is detected, the B-lane instruction is delayed by one cycle.If pair_raw0 (A reads B) occurs, it is considered illegal and is blocked immediately to preserve causality and correctness.

For cross-cycle hazards, the scoreboard adopts adestination-busy-as-loc policy—any destination register still active in an execution stage (X0–X3) is marked as busy. This prevents premature forwarding from incomplete load or mul/div operations. The nearest-producer-first forwarding strategy balances timing and performance by reducing multiplexer depth and shortening the critical path between Decode and Execute.

The refinement of stall_Fhl addresses the "double-bubble" issue that occurs after a taken branch. When brj_taken_Dhl = 1, the Fetch stage updates the PC immediately instead of being held by Decode hazards, ensuring stable IPC even in branch-heavy workloads.Overall, the design achieves correct and efficient dual-issue execution without increasing hardware complexity.

Part 3 Testing Methodology

First, a test file named test.S was created, consisting of two parts. The first part uses two addi instructions that write to the same destination register (x5) to verify whether the Scoreboard can detect a WAW hazard within the same cycle and correctly delay the issuance of the second instruction. The second part employs two independent non-ALU instructions, lui and auipc, to test whether the processor can perform dual issue in the same cycle when no dependencies exist.After assembling the test, the simulation was executed using the following command:./riscvssc-sim +exe=../tests/build/vmh/riscv-test.vmh +disasm=3.By analyzing the cycle-level disassembly output from the simulation, it was verified that riscvssc-CoreCtrl.v and riscvssc-CoreScoreboard.v correctly handle dependency detection and issue control in the dual-issue architecture, ensuring that the overall control logic and hazard management operate as intended.

```
// ------------------------------------------------------------
// A WAW Pair Test
//   I0: addi x5, x0, 1
//   I1: addi x5, x5, 2       (same rd=x5 → WAW; lane-B must stall)
//   Expect: final x5 = 3; x6 reads it; x10 marks pass/fail (0/1)
// ------------------------------------------------------------
.balign 8                 // encourage dual-fetch pairing

addi  x5, x0, 1           // I0 (lane-A candidate)
addi  x5, x5, 2           // I1 (lane-B candidate) → must be delayed 1 cycle

add   x6, x5, x0          // consume final x5
addi  x7, x0, 3
bne   x6, x7, 9f          // fail if not 3
addi  x10, x0, 0          // mark subtest A pass
j     3f

addi  x10, x0, 1          // mark subtest A fail

// small separator to avoid accidental pairing with next section

addi  x0, x0, 0
addi  x0, x0, 0
```

```
// ------------------------------------------------------------
// B Non-ALU Pair Test
//   I0: LUI   x10, 0x10010
//   I1: AUIPC x11, 0
//   Expect: both issue same cycle; x10!=0 and x11!=0; x13 marks 0/1
// ------------------------------------------------------------
.balign 8                 // encourage dual-fetch pairing

lui   x10, 0x10010        // non-ALU, independent of AUIPC
auipc x11, 0              // non-ALU, PC-relative, independent of LUI

add   x12, x10, x11       // consume both results to force visibility
beq   x10, x0, 19f
beq   x11, x0, 19f
addi  x13, x0, 0          // mark subtest B pass
j     8f

addi  x13, x0, 1          // mark subtest B fail

// Summary flag in x14: 0=both pass; 1/2 indicate which subtest failed; 3=both failed
//   bit0 <- x10 (A fail flag)
//   bit1 <- x13 (B fail flag)
add   x14, x10, x0        // x14 = A_fail
slli  x15, x13, 1         // x15 = B_fail << 1
or    x14, x14, x15
```

```
/{ 00080000 |        | -- |        |        |        |        |        }\
\{ 00080004 |        | -- | nop    | nop    | nop    | nop    | nop    }/
/{ 00080008 |        | -- |        |        |        |        |        }\
\{ 0008000c |        | -- | nop    | nop    | nop    | nop    | nop    }/
/{ 00080010 |        | -- |        |        |        |        |        }\
\{ 00080014 |        | -- | nop    | nop    | nop    | nop    | nop    }/
/{ 00080000 |  (-_-) | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }\
\{ 00080004 |  (-_-) | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }/
/{ 00080000 |  (-_-) | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }\
\{ 00080004 |  (-_-) | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }/
/{#00080008 |#addi   | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }\
\{#0008000c |#addi   | -- |  (-_-) |  (-_-) |  (-_-) |  (-_-) |  (-_-) }/
/{ 00080008 | addi   | \/ | addi   |  (-_-) |  (-_-) |  (-_-) |  (-_-) }\
\{ 0008000c | addi   | /\ | nop    |  (-_-) |  (-_-) |  (-_-) |  (-_-) }/
/{#00080010 |#add    | -- | addi   | addi   |  (-_-) |  (-_-) |  (-_-) }\
\{#00080014 |#addi   | -- | nop    | nop    |  (-_-) |  (-_-) |  (-_-) }/
/{ 00080010 | add    | \/ | add    | addi   | addi   |  (-_-) |  (-_-) }\
\{ 00080014 | addi   | /\ | nop    | nop    | nop    |  (-_-) |  (-_-) }/
/{#00080018 |#bne    | -- | addi   | add    | addi   | addi   |  (-_-) }\
\{#0008001c |#addi   | -- | nop    | nop    | nop    | nop    |  (-_-) }/
WB-B x5 @105
/{ 00080018 | bne    | \/ | bne    | addi   | add    | addi   | addi   }\
\{ 0008001c | addi   | /\ | nop    | nop    | nop    | nop    | nop    }/
/{ 00080020 | jal    | \/ | undef  | bne    | addi   | add    | addi   }\
\{ 00080024 | addi   | /\ | nop    | nop    | nop    | nop    | nop    }/
WB-B x7 @125
/{ 00080028 | nop    | \/ | undef  | undef  | bne    | addi   | add    }\
\{ 0008002c | nop    | /\ | nop    | nop    | nop    | nop    | nop    }/
/{#00080030 |#lui    | -- | nop    | undef  | undef  | bne    | addi   }\
```

```
WB-B x7 @125
/{ 00080028 | nop    | \/ | undef  | undef  | bne    | addi   | add    }\
\{ 0008002c | nop    | /\ | nop    | nop    | nop    | nop    | nop    }/
/{#00080030 |#lui    | -- | nop    | undef  | undef  | bne    | addi   }\
\{#00080034 |#auipc  | -- | nop    | nop    | nop    | nop    | nop    }/
WB-B x10 @145
/{ 00080030 | lui    | \/ | lui    | nop    | undef  | undef  | bne    }\
\{ 00080034 | auipc  | /\ | nop    | nop    | nop    | nop    | nop    }/
/{#00080038 |#add    | -- | auipc  | lui    | nop    | undef  | undef  }\
\{#0008003c |#beq    | -- | nop    | nop    | nop    | nop    | nop    }/
/{ 00080038 | add    | \/ | add    | auipc  | lui    | nop    | undef  }\
\{ 0008003c | beq    | /\ | nop    | nop    | nop    | nop    | nop    }/
/{-00080040-|-beq   -| -- | beq    | add    | auipc  | lui    | nop    }\
\{-00080044-|-addi  -| -- | nop    | nop    | nop    | nop    | nop    }/
WB-B x11 @185
/{ 00080044 |  (-_-) | -- |  (-_-) | beq    | add    | auipc  | lui    }\
\{ 00080048 |  (-_-) | -- |  (-_-) | nop    | nop    | nop    | nop    }/
/{#0008004c |#addi   | -- |  (-_-) |  (-_-) | beq    | add    | auipc  }\
\{#00080050 |#add    | -- |  (-_-) |  (-_-) | nop    | nop    | nop    }/
/{ 0008004c | addi   | \/ | addi   |  (-_-) |  (-_-) | beq    | add    }\
\{ 00080050 | add    | /\ | nop    |  (-_-) |  (-_-) | nop    | nop    }/
/{#00080054 |#slli   | -- | add    | addi   |  (-_-) |  (-_-) | beq    }\
\{#00080058 |#or     | -- | nop    | nop    |  (-_-) |  (-_-) | nop    }/
/{ 00080054 | slli   | \/ | slli   | add    | addi   |  (-_-) |  (-_-) }\
\{ 00080058 | or     | /\ | nop    | nop    | nop    |  (-_-) |  (-_-) }/
/{#0008005c |#addi   | -- | or     | slli   | add    | addi   |  (-_-) }\
\{#00080060 |#addi   | -- | nop    | nop    | nop    | nop    |  (-_-) }/
WB-B x14 @245
/{ 0008005c | addi   | \/ | addi   | or     | slli   | add    | addi   }\
\{ 00080060 | addi   | /\ | nop    | nop    | nop    | nop    | nop    }/
/{#00080064 |#csrw   | -- | addi   | addi   | or     | slli   | add    }\
\{#00080068 |#bne    | -- | nop    | nop    | nop    | nop    | nop    }/
WB-B x14 @265
/{ 00080064 | csrw   | \/ | csrw   | addi   | addi   | or     | slli   }\
\{ 00080068 | bne    | /\ | nop    | nop    | nop    | nop    | nop    }/
/{-0008006c-|-nop   -| -- | bne    | csrw   | addi   | addi   | or     }\
\{-00080070-|-nop   -| -- | nop    | nop    | nop    | nop    | nop    }/
WB-B x2 @285
/{ 00080060 |  (-_-) | -- |  (-_-) | bne    | csrw   | addi   | addi   }\
\{ 00080064 |  (-_-) | -- |  (-_-) | nop    | nop    | nop    | nop    }/
/{#00080068 |#bne    | -- |  (-_-) |  (-_-) | bne    | csrw   | addi   }\
\{#0008006c |#nop    | -- |  (-_-) |  (-_-) | nop    | nop    | nop    }/
/{-00080068-|-bne   -| \/ | bne    |  (-_-) |  (-_-) | bne    | csrw   }\
\{-0008006c-|-nop   -| /\ | nop    |  (-_-) |  (-_-) | nop    | nop    }/
*** PASSED ***
```

The simulation output clearly demonstrates that all test objectives were successfully achieved.At addresses 0x00080008 and 0x0008000c, the two addi instructions were decoded in the same cycle, but the issue stage displayed a “/” and “/\” alternation — indicating that the first instruction was issued first while the second was delayed by one cycle. This behavior confirms that the Scoreboard correctly detected the WAW hazard and postponed the B-lane issue. Subsequently, the add and bne instructions executed correctly, with the final value of x5 equal to 3.

In the latter part, the lui and auipc non-ALU instructions were decoded simultaneously and successfully issued in the same cycle. The following add x12, x10, x11 executed normally, and both registers held nonzero values, showing that dual issue worked correctly when no dependency existed**.**

Finally, the simulation output ended with *** PASSED ***, confirming that both riscvssc-CoreCtrl.v and riscvssc-CoreScoreboard.v correctly handled hazard detection and dual-issue control.

PART4 Evaluation

bench	core	num_cycle	num_inst	ipc
vvadd	riscvlong	1447	1064	0.735314
vvadd	riscvlong(rand)	9786	1063	0.108625
vvadd	dualfetch	1961	88	0.044875
vvadd	dualfetch(rand)	13665	18	0.001317
vvadd	riscvssc	(timeout)		
vvadd	riscvssc (rand)	(timeout)		
cmplx-mult	riscvlong	16685	2949	0.176746
cmplx-mult	riscvlong(rand)	39834	2948	0.074007
cmplx-mult	dualfetch	5169	17	0.003289
cmplx-mult	dualfetch(rand)	36055	4	0.000111
cmplx-mult	riscvssc	(timeout)		
cmplx-mult	riscvssc(rand)	(timeout)		
Masked-filter	riscvlong	17701	6964	0.393424
Masked-filter	riscvlong(rand)	61225	6962	0.113712
Masked-filter	dualfetch	13625	694	0.050936
Masked-filter	dualfetch(rand)	54441	98	0.001800
Masked-filter	riscvssc	(timeout)		
Masked-filter	riscvssc(rand)	(timeout)		
Bin-search	riscvlong	1539	1126	0.731644
Bin-search	riscvlong(rand)	5584	1124	0.201289
Bin-search	dualfetch	2127	118	0.055477
Bin-search	dualfetch(rand)	9264	31	0.003346
Bin-search	riscvssc	159	63	0.396226
Bin-search	riscvssc(rand)	540	48	0.088889

PART5 Discussion

This experiment compares the performance of three processor architectures: riscvlong, riscvdualfetch, and riscvssc. The results show that riscvlong demonstrates the most stable performance across all tests, with an IPC ranging from approximately 0.4 to 0.7. Although riscvdualfetch adopts a dual-fetch design that should theoretically enhance instruction-level parallelism, its actual IPC drops significantly due to incomplete hazard detection and pairing control. riscvssc encounters timeouts in most benchmarks because of synchronization and dependency control issues, but it still operates correctly and shows potential in simpler workloads such as bin-search. Overall, the main performance bottleneck lies in the variation of the Cycles Per Instruction (CPI) — the second term in the Iron Law of Processor Performance. With improved implementations of the scoreboard, bypass, and dual-issue control mechanisms, the riscvssc architecture could achieve a substantial boost in overall processing efficiency.

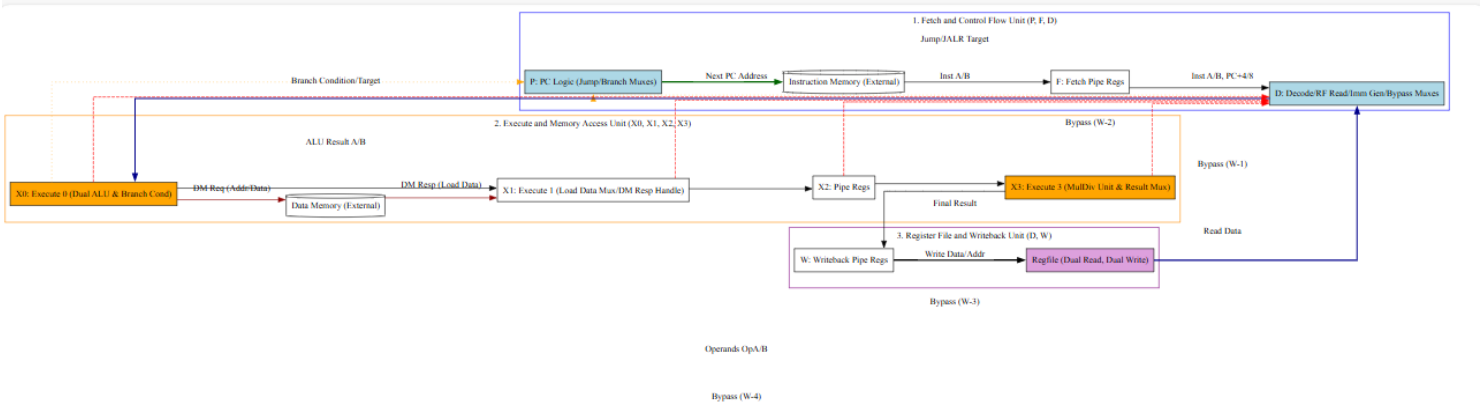

Figure1. Datapath diagram of heterogeneous dual-issue superscalar I4 processor

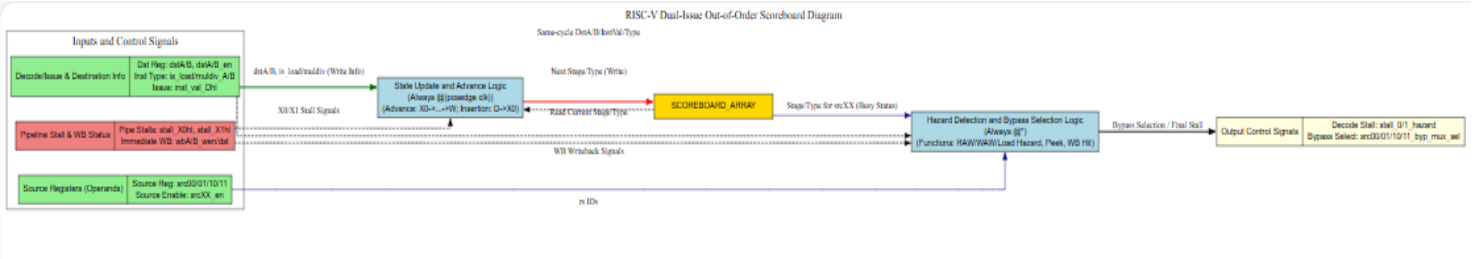

Figure2. Diagram of the implemented scoreboard